

Silent Guardian: Protecting Text From Malicious Exploitation by Large Language Models

Jiawei Zhao^{ID}, Kejiang Chen^{ID}, *Member, IEEE*, Xiaojian Yuan, Yuang Qi, Weiming Zhang^{ID}, and Nenghai Yu^{ID}

Abstract—The rapid development of large language models (LLMs) has yielded impressive success in various downstream tasks. However, the vast potential and remarkable capabilities of LLMs also raise new security and privacy concerns if they are exploited for nefarious purposes due to their open-endedness. For example, LLMs may be used to plagiarize or imitate writing, thereby infringing the copyright of the original content or to create indiscriminate fake information based on a certain source text. In some cases, LLMs can even analyze text from the Internet to infer personal privacy. Unfortunately, previous text protection research could not foresee the emergence of powerful LLMs, rendering it no longer effective in this new context. To bridge this gap, we introduce *Silent Guardian (SG)*, a text protection mechanism against LLMs, which allows LLMs to refuse to generate responses when receiving protected text, preventing the malicious use of text from the source. Specifically, we first propose the concept of *Truncation Protection Examples (TPE)*. By carefully modifying the text to be protected, TPE can induce LLMs to first sample the end token, thus directly terminating the interaction. In addition, to efficiently construct TPE in the discrete space of text data, we propose a novel optimization algorithm called *Super Tailored Protection (STP)*, which is not only highly efficient but also maintains the semantic consistency of the text during the optimization process. The comprehensive experimental evaluation demonstrates that SG can effectively protect the target text under various configurations and achieve almost 100% protection success rate in some cases. Notably, SG also exhibits relatively good transferability and robustness, making its application in practical scenarios possible. Our code is available at <https://github.com/weiyezhimeng/Silent-Guardian>.

Index Terms—Text protection, silent guardian, truncation protection example, large language model.

I. INTRODUCTION

RECENT advances in large language models (LLMs) have led to impressive performance in a variety of downstream language tasks [1], such as holding natural conversation [2], text and code generation [3], [4], log anomaly detection [5] and reading comprehension [6]. By automating tedious tasks

and readily providing comprehensive information, they are expected to increase the productivity of society dramatically. However, while bringing convenience to people, this powerful ability also creates new security and privacy risks. For example, malicious users can use LLMs to exploit internet text to engage in illegal activities. Recent research [7] has indicated that by leveraging individual statements from social media, LLMs such as GPT-4 can accurately infer personal information such as gender, income, and location. This exposes personal privacy to enormous risks. In addition, the emergence of LLMs has automated the process of plagiarists and specialized artificial intelligence article rotation tools already exist [8], which poses a major challenge to copyright protection. Additionally, the capability of LLMs to mass-produce targeted rumors based on source texts [9] also makes governance in cyberspace increasingly difficult. It is worth noting that when LLMs have the ability to actively retrieve Internet texts, e.g., Bing Chat, highly automated malicious behaviors under the instructions of malicious users become possible, and the above risks are further amplified.

Therefore, there is an urgent need for a text protection mechanism to address these new risks in the context of LLMs to prevent text from being exploited maliciously. However, previous text protection methods mainly focused on copyright and privacy protection, each with its own shortcomings.

For text copyright protection, typical methods involve embedding watermark information into the text. This includes traditional synonym substitution-based methods [10], [11] and recently developed generative model-based watermarking methods [12], [13]. However, watermarking methods, being a passive defense, can only respond after an attack has occurred. Instead, some methods encode the content into Unicode, making it impossible for third parties to replicate it [14]. However, methods that restrict document copying can be easily bypassed by Optical Character Recognition (OCR).

For text privacy protection, existing methods mainly ensure the privacy of user text by changing the computation method or environment, such as fully homomorphic encryption [15], trusted execution environments [16], secure multi-party computation [17], and differential privacy [18]. However, fully homomorphic encryption requires excessive computational overhead, trusted execution environments, and secure multi-party computation, limited by communication and physical hardware constraints. Differentiated privacy may sacrifice the utility of the data in some cases.

Aiming to bridge these gaps, we propose a novel text protection mechanism against LLMs called *Silent Guardian*

Received 18 January 2024; revised 4 June 2024 and 27 June 2024; accepted 12 July 2024. Date of publication 6 September 2024; date of current version 26 September 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62121002, Grant U20B2047, Grant U2336206, Grant 62102386, Grant 62472398, and Grant 62372423; and in part by the Open Foundation of Key Laboratory of Cyberspace Security, Ministry of Education under Grant KLCS20240207. The associate editor coordinating the review of this article and approving it for publication was Prof. Kwok-Yan Lam. (*Corresponding author: Kejiang Chen.*)

The authors are with the CAS Key Laboratory of Electromagnetic Space Information, Anhui Province Key Laboratory of Digital Security, School of Information Science and Technology, University of Science and Technology of China, Hefei 230026, China (e-mail: chenkj@ustc.edu.cn).

Digital Object Identifier 10.1109/TIFS.2024.3455775

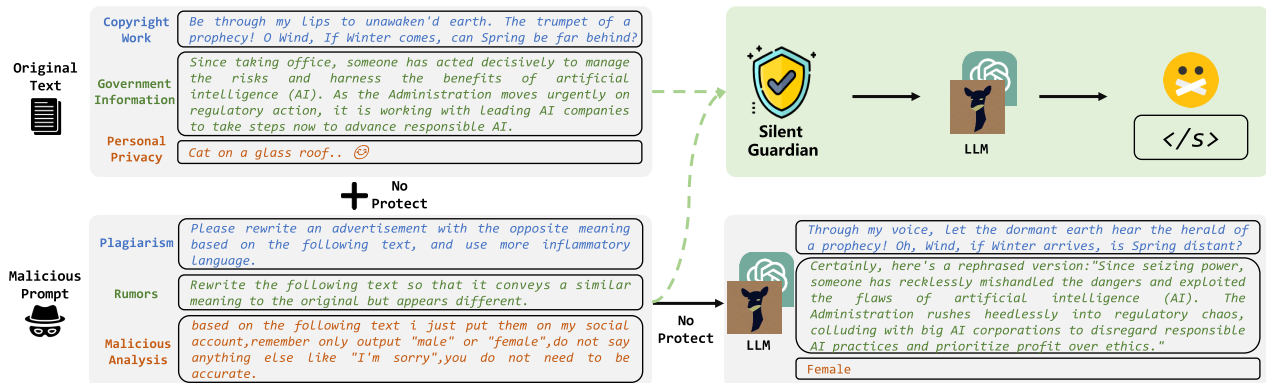


Fig. 1. **Scenario of Silent Guardian.** The adversary, upon acquiring the target text, articulates their requirements by adding a prompt to the original text, thereby leading the model to produce harmful results. In the Silent Guardian mechanism, STP aims to fine-tune the original text to prevent LLMs from generating any response. This kind of text is called TPE in this paper. The black arrows depict the adversary's workflow, exemplifying three types of malicious operations: plagiarism, rumor fabrication, and malicious analysis, corresponding to copyrighted works, government information, and personal privacy, respectively. The green arrows represent the protective process of SG.

(SG), which can convert a piece of the original text to protected text. Generally, when protected text is fed into LLMs as part of a prompt by malicious users, it will silence the LLMs, i.e., prevent them from generating any response and simply terminate the current conversation. We refer to such protected text as *Truncation Protection Examples (TPE)*. Figure 1 provides an illustration of SG.

A. Intuition

We mainly focus on auto-regressive language models, e.g., the GPT series, which generate a probability distribution for the next token after receiving a prompt. Then, following a sampling method to obtain a specific token, this token is appended to the prompt for the next round of generation. This process will be repeated until one round of sampling results is a specific type of token known as the “end token”. Furthermore, prior research [19], [20], [21], [22], [23] has shown that LLMs may inherit the vulnerability of language models to adversarial examples [24], [25], [26], [27], [28], [29], [30]. When well-designed input text, i.e., an adversarial example for LLM, is fed into the LLMs, they can be induced to generate target content.

Based on the above intuition, if the protected text can induce the LLMs to always sample the “end token” in the first round, then they will not be able to generate subsequent answers. Specifically, our text protection involves three stages: the first stage calculates the negative log of the first round's end token probability as the loss function and backpropagates to obtain gradients. In the second stage, using the gradients from the first stage, we construct replacement sets for each token in the text. In the third stage, the results are fed forward into the model to find the optimal text as the starting point for the next round. We refer to this text protection method as *Super Tailored Protection (STP)*. Figure 2 provides an example of TPE constructed by STP.

The main contributions of this paper can be summarized as follows:

- We propose SG, the first text protection mechanism to prevent the malicious utilization of LLM, providing protection for the privacy and copyright of user-uploaded internet text.

- We present the first method for realizing SG called STP. Compared to previous optimization methods, STP offers efficient optimization while maintaining a certain degree of concealment. Additionally, its implementation of concealment does not require any inference model, making it highly scalable.
- We conducted experiments on different lengths and types of text on the LLaMA, Vicuna, and Guanaco models, demonstrating the comprehensiveness and effectiveness of the STP method.

II. RELATED WORK AND PRELIMINARY

In this section, we will review previous studies on traditional text protection, adversarial examples, and adversarial prompt against LLMs. Then, we will introduce some notations of LLM in this paper.

A. Traditional Text Protection

Previous work on text protection mainly focused on two aspects: copyright protection and privacy protection of the text.

For text copyright protection, typical methods involve embedding watermark information into the text. This includes traditional synonym substitution-based methods [10], [11] and recently developed generative model-based watermarking methods [12], [13]. However, watermarking methods, being a passive defense, can only respond after an attack has occurred. Instead, some methods encode the content into Unicode, making it impossible for third parties to replicate it [14]. However, methods that restrict document copying can be easily bypassed by Optical Character Recognition (OCR). The SG method is also an active defense method, but it generates adversarial content to prevent malicious analysis, making it less susceptible to perturbation by OCR or other methods, thus providing better protection.

For text privacy protection, existing methods mainly ensure the privacy of user text by changing the computation method or environment, such as fully homomorphic encryption [15], trusted execution environments [16], secure multi-party computation [17], and differential privacy [18]. However, fully homomorphic encryption requires excessive computational

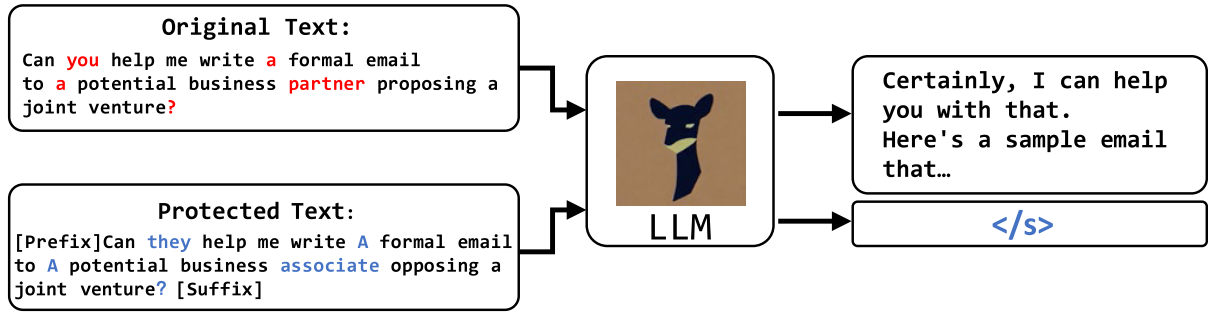


Fig. 2. An example of constructing TPE using STP on Vicuna. `</s>` represents the end token. After the token replacements shown in the box, this text successfully led the model to select the end token in the first sampling round. It can be observed that in the TPE constructed by STP, the model autonomously selects replacements such as letter casing changes and morphologically similar symbols (‘?’ to ‘?’). “[Prefix]” and “[Suffix]” represent additional requests that malicious users might add.

overhead, trusted execution environments, and secure multi-party computation, limited by communication and physical hardware constraints. Differentiated privacy may sacrifice the utility of the data in some cases. In contrast, SG can be performed in offline scenarios without the need for real-time computation. The computational overhead is acceptable in practical scenarios, and the environment deployment is simple and user-friendly. Compared to differential privacy, it also offers higher protection of text quality.

B. Adversarial Examples

1) *Definition*: Szegedy et al. [31] initially introduced adversarial examples for computer vision applications. Let $H : \mathcal{X} \rightarrow \mathcal{Y}$ be a classifier, where $\mathcal{X}$ and $\mathcal{Y}$ are the input and output domains, respectively. Assuming $x \in \mathcal{X}$ is an input to the model, the model’s prediction is denoted as $y = H(x) \in \mathcal{Y}$, and an adversarial example is $x' \in \mathcal{X}$ such that $H(x') \neq y$ belongs to a specified class. Additionally, the distance between x and x' should be as close as possible. Let $\rho : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}^+$ represent a distance metric. Setting a threshold ϵ , $\rho(x, x') < \epsilon$ serves as a measure of imperceptibility. Given a loss function ℓ , the problem of constructing adversarial examples can be formulated as an optimization problem:

$$\min_{x' \in \mathcal{X}} \ell(x', y; H) \quad \text{subject to } \rho(x, x') < \epsilon \quad (1)$$

2) *Textual Adversarial Examples*: However, the optimization problem in Equation 1 has been widely applied to continuous data such as images and speech. It does not directly apply to text data because the data space X is discrete, and the distance metric ρ is difficult to define for text data. To circumvent these two issues, several attack algorithms at the character level, word level, and sentence level have been proposed. Character-level methods [24], [25] typically adjust characters through operations such as insertion, deletion, and swapping. Word-level methods create adversarial examples through word replacement, insertion, or deletion, using techniques like synonym replacement [26], [32], replacement with words close in embedding space [27], or leveraging language models to find the best replacement [28]. Some methods also focus on inserting or deleting words to construct adversarial examples [29]. Sentence-level methods perform extensive modifications at the sentence level [30], which can effectively disrupt model outputs but are less covert.

3) *Adversarial Examples for Protection*: Some previous work has attempted to use adversarial examples for positive scenarios, focusing primarily on safeguarding the privacy of images [33], [34] or text [35]. The goal is to interfere with the results of the model inferring privacy attributes, thereby defending against inference attacks and protecting privacy. However, these methods only consider classification models and fail when facing generative models with more complex outputs. In addition, unlike classification models that can directly perturb classification results, determining the perturbation effects on generative models is also an important issue.

C. Adversarial Prompt Against LLMs

With pre-trained language models [36], [37] becoming mainstream, prompt engineering [38] has become increasingly popular in recent years. However, recent research shows that through carefully constructed adversarial prompts, language models, including LLMs, can be induced to output specified content. To achieve this, Autoprompt [19], GCG [20], and UAT [21] perform a greedy search to optimize the combination of tokens. PEZ [23] directly optimizes from the initial text, while GBDA [22] considers the adversarial example’s stealthiness and fluency but requires the introduction of additional models for constraints. These works explore classification tasks such as sentiment analysis and natural language inference, as well as generative tasks such as red team testing and target generation.

D. Notations

Given a token sequence $[x_1, x_2, \dots, x_n] \in \mathcal{V}^n$, where $\mathcal{V} = \{token_1, token_2, \dots, token_V\}$ represents the set composed of all tokens in the vocabulary. V and n denote the size of the model’s vocabulary and the length of the token sequence, respectively.

A simple sequence of tokens cannot be processed by LLM, so each x_i should be mapped to a vector before being input into LLM. To achieve this, we represent each x_i as a one-hot vector $v_i \in \mathbb{R}^V$ and pass it through a pre-trained lookup table M_e to obtain the final vector representation of token sequence, i.e.,

$$[v_1 M_e, v_2 M_e, \dots, v_n M_e] \in \mathbb{R}^{n \times d}, \quad (2)$$

where d refers to the dimension of the embedding vector. After inputting the above result into LLM, the output of the LLM logits layer $g \in \mathbb{R}^V$ will be obtained, and after normalization, it can be used as a prediction of the probability distribution of the next token. For simplicity, we can use:

$$p(x_{n+1} | x_1, x_2, \dots, x_n) \quad \forall x_{n+1} \in V \quad (3)$$

to represent the probability distribution for x_{n+1} . This can be denoted simply as $p(x_{n+1} | x_{<n+1})$.

After providing the probability prediction as described above, LLM can determine the next token x_{n+1} through different sampling methods and then add this token to the original token sequence to obtain a new token sequence $[x_1, x_2, \dots, x_n, x_{n+1}]$. LLM will repeat this process until a special token, the end token, is sampled.

Therefore, given a prompt $\mathcal{P}$ and the corresponding model's response as $r = [r_1, r_2, \dots, r_{\text{stop}}] \in R_{\mathcal{P}}$, the probability distribution for r can be represented as:

$$\begin{aligned} p(r | \mathcal{P}) &= p(r_1 | \mathcal{P}) \cdot p(r_2 | \mathcal{P}, r_1) \cdot \dots \cdot p(r_{\text{stop}} | \mathcal{P}, r_{<\text{stop}}) \\ &= \prod_{i=1}^{\text{stop}} p(r_i | \mathcal{P}, r_{<i}). \end{aligned} \quad (4)$$

Here, r_{stop} represents the end token. $R_{\mathcal{P}}$ represents the set of all answers given by LLM to $\mathcal{P}$, $\sum_{r \in R_{\mathcal{P}}} p(r | \mathcal{P}) = 1$.

III. THREAT MODEL

To be more practical, Silent Guardian needs to meet the following three requirements:

- 1) *Stealthiness*: Modifications to the protected text must be imperceptible to humans, to retain its semantic information and high readability as much as possible.
- 2) *Disruptiveness*: Protected text cannot be effectively analyzed and exploited by LLMs, meaning that LLMs cannot generate any response to the protected text in our scenario.
- 3) *Scalability*: It should be able to handle text of various lengths to cope with different malicious scenarios.

While meeting the aforementioned requirements, we considered two different LLM scenarios:

- 1) The architectures and parameters of the target LLMs are accessible, e.g., the open-source LLMs.
- 2) Only the scope of the target LLMs is known.

Silent Guardian should demonstrate excellent performance in the first scenario and can exhibit promising performance in the second challenging scenario.

IV. SILENT GUARDIAN

Existing text protection work cannot effectively address the issue of malicious exploitation of text by LLMs. To cope with this scenario, we introduce Silent Guardian (SG), a text protection mechanism against LLMs. The workflow of SG is to fine-tune the text to be protected as a Truncation Protection Example (TPE) to prevent malicious exploitation by LLMs. Therefore, in this section, we will first introduce TPE and then propose a novel algorithm to efficiently construct TPE, called Super Tailored Protection (STP).

A. Truncation Protection Example

TPE is the protected text that can silence the LLMs, i.e., prevents them from generating any response and simply terminates the current conversation. To construct TPE, We can formalize the objective of constructing TPE as finding the minimum value of a loss function. Since the characteristic of TPE, an intuitive loss function would be the expected length of the model's response.

For the input $\mathcal{P}$, let the answer that selected end token in the first round of sampling be $r_e = [\text{end token}]$, $R_{\text{remain}} = R_{\mathcal{P}} - r_e$. Then, we can define this loss function as:

$$\begin{aligned} \mathcal{L}_{TPE}(\mathcal{P}) &= \sum_{r \in R_{\mathcal{P}}} p(r | \mathcal{P}) \cdot \text{len}(r) \\ &= p(r_e | \mathcal{P}) \cdot 1 + \sum_{r \in R_{\text{remain}}} p(r | \mathcal{P}) \cdot \text{len}(r), \end{aligned} \quad (5)$$

where $\text{len}(r)$ denotes the number of tokens in r .

It is a challenging problem to find a $\mathcal{P}$ that minimizes $\mathcal{L}_{TPE}$ in Equation 5. However, we notice that by maximizing $p(r_e | \mathcal{P})$, $\mathcal{L}_{TPE}$ can reach its minimum value of 1. Therefore, we can transform the problem into optimizing $p(r_e | \mathcal{P})$ to achieve the maximum value, and the final loss function can be represented as:

$$\mathcal{L}_{TPE}(\mathcal{P}) = -\log(p(r_e | \mathcal{P})), \quad (6)$$

and then we can convert the goal of constructing TPE into an optimization problem:

$$\arg \min_{\mathcal{P}' \in \text{constraint}(\mathcal{V})^{\text{len}(\mathcal{P})}} \mathcal{L}_{TPE}(\mathcal{P}'), \quad (7)$$

where “ $\text{constraint}(\mathcal{V})$ ” refers to the constraint imposed on the available tokens for selection.

B. Super Tailored Protection

With the formalized objective of constructing TPE, in this section, we will introduce an effective and stealthy method called Super Tailored Protection (STP) to achieve this.

Figure 3 illustrates the overview of STP. The STP method comprises two modules. In the first module, we represent the text to be protected using one-hot vectors, define the loss function $\mathcal{L}_{TPE}$, and compute gradients of one-hot vectors. In the second module, we construct suitable replacement candidate sets, referred to as $\text{constraint}(\mathcal{V})$, using gradients that we have computed in the first module. Then, we utilize greedy search to identify the optimal replacements that minimize the loss function. The detailed process is shown below.

1) *Representation of the Prompt Using One-Hot Vectors*: Given the prompt to be optimized, denoted as $\mathcal{P}$, let each token composing $\mathcal{P}$ be denoted as $\mathcal{P}_i$. We represent $\mathcal{P}_i$ as a one-hot vector $v_i \in \mathbb{R}^V$

$$\mathcal{P} = [v_1 M_e, v_2 M_e, \dots, v_l M_e]. \quad (8)$$

2) *Defining Loss Function*: We define the loss function as the cross-entropy between the probability distribution of the first token predicted by the LLM and the probability distribution where the end token has a probability of 1. Specifically, we utilize the output of the LLM logits layer g

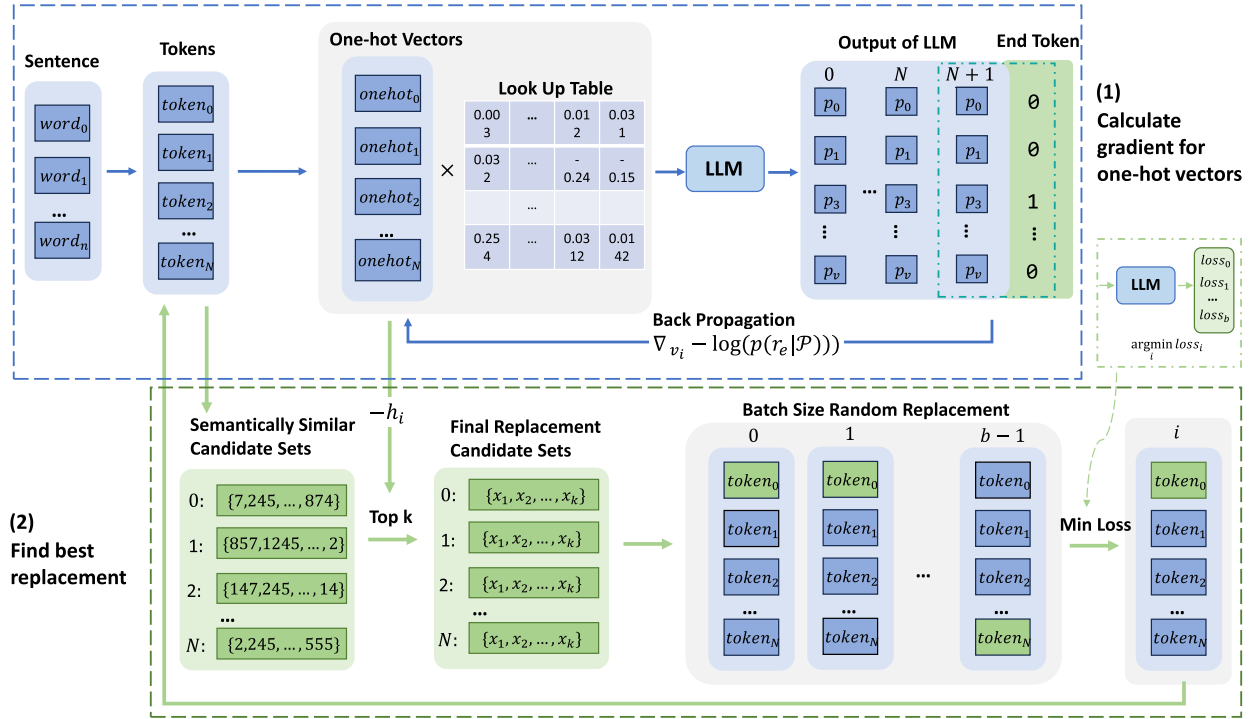


Fig. 3. **The overview of Super Tailored Protection.** (1) Calculate gradient for one-hot vectors: Convert the text to be protected into a one-hot vector representation. Input this into the LLM and utilize the probability distribution of the predicted N+1th token and the end token's probability distribution to compute the loss function. Calculate the gradient and propagate it backward. (2) Find the best replacement: Initially, generate a semantically similar candidate set for each token in the text to be protected using neighboring tokens from the embedding layer. Then, take the results from step 1 to construct the final replacement candidate set from the semantically similar candidate set. Lastly, randomly select and identify the best replacement as the starting text for the next iteration.

and the one-hot vector of the end token v_{end} for computation, i.e.,

$$\text{loss} = H(g, v_{\text{end}}), \quad (9)$$

It is worth noting that selecting different loss functions enables the STP method to achieve diverse objectives, showcasing the algorithm's versatility.

3) *Gradient Backpropagation*: Computing the gradient of the one-hot vector corresponding to p_i ,

$$h_i = -\nabla_{v_i} \text{loss} \in \mathbb{R}^V. \quad (10)$$

Each dimension of h_i corresponds to a token in $\mathcal{V}$, denoted as $h_i[j]$, where $j \in \{1, 2, \dots, V\}$. A smaller $h_i[j]$ indicates that replacing $\mathcal{P}_i$ with token_j would have a larger impact on the loss function, making it converge faster.

4) *Construction of Semantically Similar Candidate Sets*: To find semantically similar tokens, we will utilize embeddings [27] to find tokens close in the embedding layer. For each $\mathcal{P}_i$, select n closest tokens from $\mathcal{V}$ to construct a semantically similar candidate set. Specifically, We first represent all tokens in the dictionary $\mathcal{V}$ as embedding vectors and normalize them using the ℓ_2 norm to obtain a new set $\mathcal{V}'$. For token $\mathcal{P}_i$, we perform the same operation, then perform dot products with all vectors in $\mathcal{V}'$, and select the n tokens with the largest results as the set of semantically similar tokens N_i .

5) *Construction of Final Replacement Candidate Sets*: To ensure that the replacement maintains similarity with the protected text while causing the loss function to decrease,

our final replacement set is selected from within N_i by h_i . Specifically, For each token $\text{token}_j \in N_i$, sort them by the $h_i[j]$ values in descending order. Choose the top k tokens as the final replacement set, denoted as $S_i = \text{Top-k}(N_i)$.

6) *Random Replacement and Greedy Search*: In order to accommodate longer lengths of protected text, we employ a combination of random replacement and greedy search to find an optimized prompt. The specific approach is outlined as follows. In each iteration, repeat $\mathcal{P}$ *batch size* times and we can obtain an initial set $I = \{\tilde{\mathcal{P}}^1, \tilde{\mathcal{P}}^2, \dots, \tilde{\mathcal{P}}^{\text{batch size}}\}$, $|I| = \text{batch size}$. Next, we need to construct a new optimized prompt set, I' . Each $\tilde{\mathcal{P}}^i \in I$ needs to change the token in one position compared with the original $\mathcal{P}$ to construct it. The specific method is as follows:

First, if $\text{batch size} > \text{len}(\mathcal{P})$, divide I into $\text{len}(\mathcal{P})$ parts, $I_1, I_2, \dots, I_{\text{len}(\mathcal{P})}$, each part corresponding to one changed position i . If $\text{batch size} \leq \text{len}(\mathcal{P})$, randomly select a unique position i for each $\tilde{\mathcal{P}}^j \in I$.

Second, Randomly select tokens from S_i for these positions to perform random replacements, which reduces time consumption for long protected text. Specifically, for $\tilde{\mathcal{P}} \in I_i$, let

$$\tilde{\mathcal{P}}_i = \text{Uniform}(S_i). \quad (11)$$

Third, Compute the minimum loss replacement among these prompts in each iteration to obtain the new prompt $\mathcal{P}$. Repeat this process for a specified number of iterations.

The steps described above are presented in Algorithm 1.

Algorithm 1 Super Tailored Protection**Input:** Original Prompt $\mathcal{P}$, Iterations T , Loss Function $\mathcal{L}$, Batch Size B **Output:** Optimized prompt $\mathcal{P}$ **repeat** T times loss = $\mathcal{L}(\mathcal{P})$ **for** $i = 1, \dots, \text{len}(\mathcal{P})$ **do** $h_i = -\nabla_{v_i} \text{loss}$ $N_i = N(\mathcal{P}_i)$ $S_i = \text{Top} - k(N_i)$ $\text{len}_{\text{part}} = \frac{B}{\text{len}(\mathcal{P})}$ **for** $b = 1, \dots, B$ **do** **if** $B > \text{len}(\mathcal{P})$ **then** $i = \lceil \frac{b}{\text{len}_{\text{part}}} \rceil$ **else** **repeat** $i = \text{random}(1, \text{len}(\mathcal{P}))$ **until** $i \neq \text{previous } i$ $\tilde{\mathcal{P}}^{(b)} = \mathcal{P}$ $\tilde{\mathcal{P}}_i^{(b)} = \text{Uniform}(S_i)$ $\mathcal{P} = \tilde{\mathcal{P}}^{(b^*)}$, where $b^* = \arg \min_b \mathcal{L}(\tilde{\mathcal{P}}^{(b)})$ **return** $\mathcal{P}$

V. EXPERIMENTS AND EVALUATION

A. Setup

1) *Dataset*: In theory, STP does not impose specific requirements on the theme or content of the prompt itself. To comprehensively validate the effectiveness of the defense, we selected 80 prompts across 9 different categories from Vicuna official website [39], which include writing, roleplay, common-sense, fermi, counterfactual, coding, math, generic, and knowledge. We denote this data set as the Vicuna dataset.

In addition, we selected a set of texts from the novel *Warden* with varying lengths to verify the effectiveness of the text protection method on different text lengths. Specifically, we constructed eight sets of texts of different lengths, each of which is about 40, 80, 120, 160, 200, 240, 280, and 320 tokens long. Each group has 10 texts, totaling 80 texts. We denote this data set as the Novel dataset.

2) *Model*: To demonstrate the effectiveness of the STP, we conducted experiments on three transformer architecture models. These models are LLaMA [40], Vicuna v1.3 [39], and Guanaco [41] in the 7B version. The training of the last two models was built upon the LLaMA model. Specifically, Vicuna was fine-tuned on LLaMA by SFT, while Guanaco was fine-tuned on LLaMA by QLoRA.

Because constructing TPE using the STP method requires model parameters and the transferability of TPE relies on similar model architectures, we did not conduct experiments on non-open-source GPT series models in our study.

3) *Perparameters*: In this paper, the main parameters are the batch size and the number of elements in the sets N_i and S_i , which are all related to the size of the search space, and the latter two are also related to the concealment of the TPE. After

finding a trade-off, we selected *batch size* = 1024, $|N(i)|$ = 10, and $|S(i)|$ = 5 to achieve better results. Additionally, we set the number of epochs to $T = 15$. More epochs imply a higher probability of selecting the end token but also result in increased computational overhead.

4) *Metrics*: We propose three metrics to measure the effectiveness of text protection against LLMs. These metrics are the Character Replacement Ratio γ , Semantic Preservation η , and the Success Rate of Truncation Protection (PSR).

1. γ , the Character Replacement Ratio, measures the minimum number of characters changed to achieve a certain level of truncation protection. A smaller γ value indicates better concealment because fewer characters are altered. We calculate γ using the Vladimir Levenshtein edit distance [42] divided by the original sentence's character length.
2. η , Semantic Preservation, quantifies the semantic similarity between two sentences before and after token replacement. A higher η value suggests that truncation protection has a smaller impact on the sentence's meaning. We utilized the cosine similarity of sentences [43] to this metric.
3. PSR, Success Rate of Truncation Protection, represents the effectiveness of truncation protection. We define PSR as the softmax score corresponding to the end token, i.e.,

$$\text{PSR} = \frac{e^{z_{\text{end}}}}{\sum_{i=1}^V e^{z_i}}, \quad (12)$$

where z_i represents the value in the model's logits layer corresponding to *token_i*. A higher PSR indicates a more successful protection effectiveness.

These metrics help evaluate the quality of text protection and its impact on both the text's semantics and the extent to which it effectively truncates the model's output.

5) *Baseline*: We used GBDA [22] and PEZ [23] methods as baselines. Equation 6 presents the optimization objective of STP. For baseline, we employed both PEZ and GBDA to achieve this optimization objective. The optimized initial value is set to the text to be protected. Then, we computed the loss function in Equation 6 and utilized PEZ and GBDA algorithms individually to optimize the prompt. It's important to note that while the GBDA method offers constraints on the concealment of adversarial examples, it requires a specific inference model. Hence, in this paper, we didn't impose concealment constraints on GBDA. Additionally, to prevent gradient explosions when using these two methods, we used the Adam optimizer with values of epsilon (eps) $1e - 5$.

B. Evaluation

1) *White Box*:

a) *The comprehensiveness of the truncation protection example*: Table I shows the results of TPE constructed by STP, PEZ, and GBDA on nine categories of prompts from the Vicuna dataset on LLaMA, Vicuna, and Guanaco. We applied 15 rounds of replacements to each of the 80 prompts, with each round signifying one token replacement of the target prompt.

The experimental results indicate that STP exhibits favorable protective effects for prompts of varying categories and

TABLE I
RESULT OF TRUNCATION PROTECTION EXAMPLE

Model	Metrics	Method	Vicuna Dataset								
			Writing	Roleplay	Common-sense	Fermi	Counterfactual	Coding	Math	Generic	Knowledge
Vicuna	γ	STP	0.27	0.26	0.21	0.21	0.36	0.37	0.46	0.37	0.24
		PEZ	0.33	0.31	0.30	0.20	0.36	0.36	0.44	0.49	0.32
		GBDA	0.84	1.00	0.92	0.90	1.00	0.97	2.04	0.95	0.86
	η	STP	0.73	0.73	0.78	0.76	0.66	0.74	0.76	0.66	0.76
		PEZ	0.75	0.73	0.72	0.78	0.71	0.76	0.79	0.62	0.69
		GBDA	0.50	0.50	0.50	0.51	0.51	0.52	0.49	0.48	0.50
	PSR	STP	0.97	0.95	0.88	1.00	0.63	0.79	0.99	0.79	0.88
		PEZ	0.05	0.05	0.07	0.05	0.08	0.03	0.06	0.07	0.10
		GBDA	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
LLaMA	γ	STP	0.26	0.31	0.24	0.21	0.44	0.40	0.28	0.37	0.27
		PEZ	0.35	0.38	0.41	0.36	0.34	0.37	0.74	0.50	0.46
		GBDA	0.86	1.00	0.91	0.92	0.98	0.92	1.86	0.92	0.88
	η	STP	0.73	0.71	0.74	0.75	0.63	0.69	0.69	0.69	0.68
		PEZ	0.73	0.69	0.66	0.69	0.73	0.72	0.65	0.61	0.64
		GBDA	0.50	0.49	0.49	0.50	0.50	0.52	0.49	0.50	0.50
	PSR	STP	0.53	0.46	0.41	0.65	0.22	0.39	0.44	0.24	0.47
		PEZ	0.05	0.04	0.03	0.02	0.03	0.01	0.03	0.04	0.02
		GBDA	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Guanaco	γ	STP	0.29	0.33	0.26	0.25	0.43	0.39	0.44	0.41	0.33
		PEZ	0.36	0.41	0.35	0.33	0.31	0.26	0.68	0.46	0.41
		GBDA	0.84	0.98	0.89	0.88	0.98	0.91	1.79	0.91	0.87
	η	STP	0.71	0.68	0.72	0.74	0.68	0.68	0.70	0.64	0.68
		PEZ	0.72	0.70	0.69	0.71	0.70	0.76	0.67	0.65	0.65
		GBDA	0.50	0.50	0.50	0.50	0.50	0.52	0.50	0.49	0.49
	PSR	STP	0.56	0.53	0.51	0.67	0.27	0.22	0.70	0.45	0.60
		PEZ	0.04	0.03	0.04	0.03	0.03	0.01	0.05	0.03	0.04
		GBDA	0.00	0.01	0.00	0.00	0.02	0.01	0.01	0.01	0.00

lengths. However, among the nine different categories of prompts tested, prompts related to Counterfactual and Math, due to their shorter lengths, experienced greater disruption from replacement, leading to relatively poorer concealment. Compared to STP, PEZ and GBDA show little to no effectiveness in constructing TPE.

To illustrate the superiority of the STP method in constructing TPE, we selected ten prompts from the “Writing” category in the Vicuna dataset. We conducted TPE construction using the STP method, PEZ, and GBDA methods on Vicuna, and the final results are depicted in Figure 4.

Furthermore, in our experiments, we observed that a significant portion of the replacement operations involved changes in capitalization, such as transforming “you” to “You” or substitutions with visually similar or identical characters but different encodings, like changing “a” to “a”. This ensured the concealment of TPE and highlighted the advantage of using tokens rather than words as the smallest replacement units. Table II displays some examples of TPE.

Moreover, due to the greedy search principle followed by the STP method, replacements that had a considerable impact on the outcome were often in fixed positions within sentences. Consequently, many rounds of replacement were focused on the same location, reinforcing the concealment of TPE.

It is important to note that conducting 15 rounds of construction did not achieve convergence in the loss function for most prompts. Therefore, conducting more rounds of construction for a single prompt should yield better protective results, but this might also lead to greater alterations in the prompt itself.

TABLE II
EXAMPLES OF TPE ON VICUNA WHILE PSR ≥ 0.95

Original Text	Protected Text
Can you help me write a resignation letter to my current employer, while leaving on good terms and expressing gratitude for the opportunities provided?	ShouldYou help me write a resignation letter to my current employer, while leaving on good terms but expressing grat attitude for the opportunities provided?
Draft an apology email to a customer who experienced a delay in their order, and provide reassurance that the issue has been resolved.	Δraft an apology contact to a customerWho experienced a delayIn their order,\r and provide reassurance that the issue has been revol.
Write a compelling product launch announcement email to inform our customers of our new software solution.	Write a comp sell product launch announcement email to inform our customers of our new software answer?.
Write a captivating movie review for a recently released science fiction film, discussing its plot, characters, and special effects.	Write a captvivating movie evalu-ate for an recently released science fiction film, discussizing its plot, character, and special effects}\$.

b) *The effectiveness of truncation protection for texts of different lengths:* With increasing prompt length, the search space for constructing TPE grows, and the concealment is enhanced under the same number of replacements. To investigate this, we conducted 15 rounds of TPE construction on the prepared Novel dataset. Table III shows the effectiveness of STP for texts of different token lengths. As expected, under the same number of rounds, the final effectiveness of TPE remains largely consistent. An improvement was even observed in

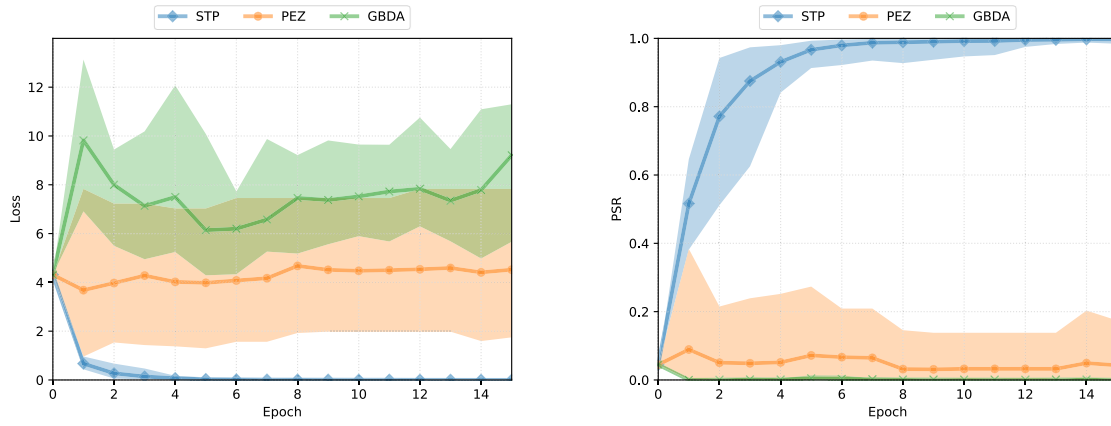


Fig. 4. The convergence results of Loss and PSR for PEZ, GBDA, and STP methods. It can be observed that our proposed approach shows faster convergence of loss and higher efficiency when it comes to constructing TPE. It is worth noting that the initial value for optimization in all three methods was set as the original prompt, and the initial steep increase in loss for GBDA is attributed to its deviation from the initial prompt in the first round, mainly due to the introduction of Gumbel-Softmax.

TABLE III
RESULT OF DIFFERENT LENGTHS OF TEXT

Model	Metrics	Novel Dataset		
		40 tokens	80 tokens	120 tokens
LLaMA	γ	0.18	0.09	0.06
	η	0.75	0.85	0.88
	PSR	0.49	0.56	0.69
Vicuna	γ	0.19	0.09	0.06
	η	0.76	0.84	0.89
	PSR	0.83	0.81	0.81
Guanaco	γ	0.18	0.09	0.07
	η	0.76	0.84	0.86
	PSR	0.57	0.66	0.67

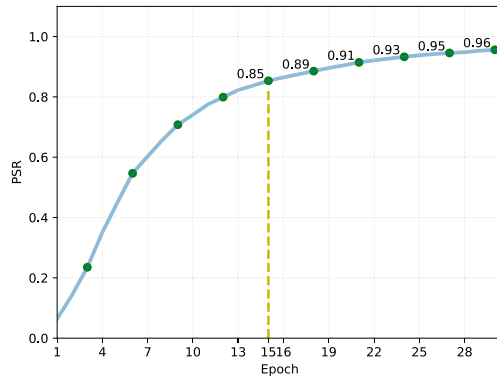


Fig. 5. Constructing TPE over 30 rounds on text of 120 tokens in the Novel dataset.

LLaMA and Guanaco. Simultaneously, the concealment of TPE increases with the length of the text. This result means that our method has a distinctive advantage in protecting long texts and is highly suitable for real-world applications.

As mentioned in section V-B1, stronger protection for a single text can be achieved by increasing the number of epochs. In this section, to control variables, we selected texts in 120 tokens from the Novel dataset and conducted 30 epochs of construction on Vicuna. Figure 5 represents the effectiveness of 30 rounds of construction.

c) *Real-world scenarios*: In real-world scenarios, adversaries often add prefixes and suffixes to texts, such as “Please summarize the following text: [exploited text]” or “[exploited

text]. Please summarize the preceding topic.” These are important steps when using LLMs for content generation. Therefore, In this section, we conducted experiments on Vicuna, LLaMA, and Guanaco by adding prefixes “Summarize following text:” “Summarize the topic of following text:” “Please summarize the topic of the following text and rewrite:” and suffixes “Summarize the preceding text.” “Summarize the topic of the preceding text.” “Please summarize the topic of the preceding text and rewrite.” to the TPE on the Novel dataset. These are denoted as prefix_1 , prefix_2 , prefix_3 , suffix_1 , suffix_2 , and suffix_3 , respectively.

The experimental results are in Table IV and Table V. The PSR and PSR* in the table, respectively, represent the Protection Success Rate of TPE before and after adding a prefix or suffix. Even though we did not specifically optimize the protection for these prefixes and suffixes, the PSR remains at a relatively high level. Furthermore, as the length of the text to be protected increases, the impact of adding prefixes and suffixes diminishes on the protective effect. Additionally, longer prefixes and suffixes have a relatively minor effect on the protective outcome. This suggests that STP exhibit superior performance in tasks involving long texts and complex prefixes/suffixes, laying the groundwork for their scalability.

2) *Transferability*: In this section, we conducted transferability experiments of TPE. Specifically, we utilized the results optimized on the Vicuna, Llama, and Guanaco models interchangeably across all three models. The experimental results are presented in Table VI and Table VII. Here, A $\rightarrow$ B denotes the transfer of optimized results from model A to model B. PSR represents the Protection Success Rate of the TPE constructed on the original model, while PSR* signifies the transferred Protection Success Rate.

In the Novel dataset, experimental results show that transferability improves with the increase in text length. However, in the Vicuna dataset, we observed that the TPE optimized for different models exhibited weaker transferability between models. To address this issue, we aggregated the loss functions of different models into a new loss function to construct TPE, hoping to enhance transferability to some extent.

TABLE IV
TRUNCATION PROTECTION EXAMPLES WITH ADDED PREFIXES

Model	Metrics	Prefix ₁			Prefix ₂			Prefix ₃		
		40 tokens	80 tokens	120 tokens	40 tokens	80 tokens	120 tokens	40 tokens	80 tokens	120 tokens
Vicuna	PSR	0.84	0.82	0.82	0.84	0.82	0.82	0.84	0.82	0.82
	PSR*	0.48	0.39	0.46	0.46	0.39	0.46	0.43	0.38	0.43
LLaMA	PSR	0.49	0.56	0.69	0.49	0.56	0.69	0.49	0.56	0.69
	PSR*	0.11	0.21	0.31	0.12	0.19	0.31	0.12	0.21	0.37
Guanaco	PSR	0.57	0.66	0.68	0.57	0.66	0.68	0.57	0.66	0.68
	PSR*	0.14	0.23	0.27	0.12	0.27	0.29	0.19	0.28	0.36

TABLE V
TRUNCATION PROTECTION EXAMPLES WITH ADDED SUFFIXES

Model	Metrics	Suffix ₁			Suffix ₂			Suffix ₃		
		40 tokens	80 tokens	120 tokens	40 tokens	80 tokens	120 tokens	40 tokens	80 tokens	120 tokens
Vicuna	PSR	0.84	0.82	0.82	0.84	0.82	0.82	0.84	0.82	0.82
	PSR*	0.29	0.39	0.37	0.25	0.33	0.39	0.53	0.51	0.56
LLaMA	PSR	0.49	0.56	0.69	0.49	0.56	0.69	0.49	0.56	0.69
	PSR*	0.11	0.15	0.12	0.15	0.20	0.19	0.21	0.28	0.25
Guanaco	PSR	0.57	0.66	0.68	0.57	0.66	0.68	0.57	0.66	0.68
	PSR*	0.21	0.15	0.22	0.21	0.18	0.27	0.38	0.40	0.52

TABLE VI
TRANSFERABILITY OF TRUNCATION PROTECTION EXAMPLES
ON NOVEL DATASET

Model	Metrics	Novel Dataset		
		40tokens	80tokens	120tokens
Vicuna→LLaMA	PSR	0.49	0.56	0.69
	PSR*	0.13	0.18	0.16
LLaMA→Vicuna	PSR	0.83	0.81	0.81
	PSR*	0.10	0.19	0.41
LLaMA→Guanaco	PSR	0.57	0.66	0.67
	PSR*	0.23	0.39	0.46
Guanaco→LLaMA	PSR	0.49	0.56	0.69
	PSR*	0.24	0.26	0.28
Vicuna→Guanaco	PSR	0.57	0.66	0.67
	PSR*	0.13	0.25	0.27
Guanaco→Vicuna	PSR	0.83	0.81	0.81
	PSR*	0.18	0.22	0.31

Specifically, for three models $model_1$, $model_2$, and $model_3$, we defined the new loss function as $(loss_{model_1} + loss_{model_2})/2$, and used this loss function to construct TPE, which is used to attack $model_3$. The specific experimental results are shown in Table VIII. Here, AVE represents the average of the results in the first two rows of the table, and AGG represents the aggregating of the other two models.

The experimental results show that the TPE constructed using the aforementioned method exhibits enhanced transferability. Compared to the original method, the average PSR values across various models in the Vicuna dataset have increased by 0.11. Furthermore, from the above experimental results, it can be reasonably inferred that aggregating the loss functions of more models to construct TPE should lead to even better transferability.

3) *The Relationship Between the Time and Hyperparameters for Constructing a Single TPE*: In the preceding sections, we analyzed the practical effectiveness of STP in text protection. In this section, we delve into the relationship between

TPE construction time and various parameters. As depicted in Figure 3, the construction time of TPE is primarily associated with the length of the text to be protected, the size of the replacement set, the number of construction rounds, and the batch size. In our experimental setup, STP for a single text on the 7B model using an NVIDIA Quadro RTX8000 GPU takes approximately 12 minutes. We will now investigate the correlation between these parameters and the construction time of TPE.

a) *Text length*: The text selected as the protection target is not an adjustable parameter. However, fortunately, due to the parallel computing nature of transformer models, the computational time for texts of different lengths remains relatively consistent. Similar to the experimental settings in Section V, we conducted 15 epochs of construction for texts in the Novel Dataset on the Vicuna-7B model. The average construction times in 40 tokens, 80 tokens, and 120 tokens text were approximately 12.95 minutes, 13.15 minutes, and 15.18 minutes, respectively.

The experimental results indicate a slight increase in construction time with an increase in the length of the protected text. Nevertheless, this increase remains within an acceptable range.

b) *Size of replacement sets*: The time complexity involved in constructing replacement sets is negligible compared to a single forward pass of the model. However, the size of the replacement set significantly influences the convergence rate of the loss function. Qualitatively, a larger semantically similar candidate set N_i broadens the model's selection scope, providing more efficient options when forming the final replacement candidate set S_i . However, it will also increase the randomness in the algorithm.

c) *Number of construction rounds*: Increasing the number of construction rounds enhances the success rate of protection but also extends the time required.

TABLE VII
TRANSFERABILITY OF TRUNCATION PROTECTION EXAMPLES ON VICUNA DATASET

Model	Metrics	Vicuna Dataset								
		Writing	Roleplay	Common-sense	Fermi	Counterfactual	Coding	Math	Generic	Knowledge
Vicuna→LLaMA	PSR	0.53	0.46	0.41	0.65	0.22	0.39	0.44	0.24	0.47
	PSR*	0.07	0.10	0.05	0.10	0.05	0.07	0.09	0.05	0.08
LLaMA→Vicuna	PSR	0.97	0.95	0.88	1.00	0.63	0.79	0.99	0.79	0.88
	PSR*	0.41	0.27	0.37	0.60	0.13	0.16	0.40	0.31	0.22
LLaMA→Guanaco	PSR	0.56	0.53	0.51	0.67	0.27	0.22	0.70	0.45	0.60
	PSR*	0.26	0.22	0.30	0.43	0.12	0.13	0.27	0.16	0.26
Guanaco→LLaMA	PSR	0.53	0.46	0.41	0.65	0.22	0.39	0.44	0.24	0.47
	PSR*	0.18	0.20	0.20	0.19	0.09	0.12	0.17	0.11	0.26
Vicuna→Guanaco	PSR	0.56	0.53	0.51	0.67	0.27	0.22	0.70	0.45	0.60
	PSR*	0.17	0.10	0.10	0.28	0.06	0.05	0.14	0.06	0.11
Guanaco→Vicuna	PSR	0.97	0.95	0.88	1.00	0.63	0.79	0.99	0.79	0.88
	PSR*	0.26	0.38	0.37	0.62	0.18	0.13	0.26	0.25	0.30

TABLE VIII
ENHANCEMENT OF TPE TRANSFERABILITY ON THE VICUNA DATASET

Model	Metrics	Vicuna Dataset								
		Writing	Roleplay	Common-sense	Fermi	Counterfactual	Coding	Math	Generic	Knowledge
Guanaco	PSR	0.56	0.53	0.51	0.67	0.27	0.22	0.70	0.45	0.60
LLaMA→Guanaco	PSR*	0.26	0.22	0.30	0.43	0.12	0.13	0.27	0.16	0.26
Vicuna→Guanaco	PSR*	0.17	0.10	0.10	0.28	0.06	0.05	0.14	0.06	0.11
\	AVE	0.21	0.19	0.20	0.35	0.09	0.09	0.20	0.11	0.18
AGG→Guanaco	PSR*	0.22↑	0.28↑	0.26↑	0.30	0.20↑	0.16↑	0.40↑	0.31↑	0.40↑
LLaMA	PSR	0.53	0.46	0.41	0.65	0.22	0.39	0.44	0.24	0.47
Vicuna→LLaMA	PSR*	0.07	0.10	0.05	0.10	0.05	0.07	0.09	0.05	0.08
Guanaco→LLaMA	PSR*	0.18	0.20	0.20	0.19	0.09	0.12	0.17	0.11	0.26
\	AVE	0.12	0.15	0.12	0.14	0.07	0.09	0.13	0.08	0.17
AGG→LLaMA	PSR*	0.24↑	0.25↑	0.26↑	0.20↑	0.15↑	0.18↑	0.25↑	0.21↑	0.32↑
Vicuna	PSR	0.97	0.95	0.88	1.00	0.63	0.79	0.99	0.79	0.88
LLaMA→Vicuna	PSR*	0.41	0.27	0.37	0.60	0.13	0.16	0.40	0.31	0.22
Guanaco→Vicuna	PSR*	0.26	0.38	0.37	0.62	0.18	0.13	0.26	0.25	0.30
\	AVE	0.33	0.32	0.37	0.61	0.15	0.14	0.33	0.28	0.26
AGG→Vicuna	PSR*	0.41↑	0.41↑	0.50↑	0.48	0.28↑	0.44↑	0.48↑	0.44↑	0.54↑

d) *Batch size*: Larger batch sizes imply a larger selection space, leading to a greater reduction in the loss function within a single round of construction. However, it also results in increased construction time.

After discussing the impact of the aforementioned parameters on construction time, in order to maintain semantic similarity, we conducted experiments by selecting a set of data near the original $|N(i)|$ and $|S(i)|$ to find more optimal parameters for reducing the time cost of constructing TPE. Specifically, we set $|N(i)| \in \{8, 10, 12\}$, $|S(i)| \in \{4, 5, 6\}$, and $batch\ size \in \{16, 32, 64, 128, 256, 512\}$. We then selected a text from the Vicuna dataset for experimentation, terminating training when the text's PSR reached or exceeded 90%, and recorded the time taken for the calculations. The experimental results are depicted in Figure 6.

It can be observed that reducing the batch size significantly decreases the running time of STP, with the optimal scenario taking only about 6 seconds. Similarly, adjusting other

parameters properly can also significantly reduce the running time of STP when a higher PSR is required.

4) *Discussion on the Relationship Between TPE Construction Time and Text Length*: In this section, we will discuss the relationship between the time required for TPE construction and the length of the text. Specifically, we conducted experiments on texts of lengths 40, 80, 120, 160, 200, 240, 280, and 320 tokens from the Novel dataset. The experimental parameters were set as $|S_i| = 5$, $|N_i| = 10$, $batch\ size = 128$, $T = 15$. The experimental results are shown in Figure 7. All experiments in this section were completed using an NVIDIA RTX A6000 GPU.

The experimental results indicate that the time cost is approximately linearly related to the length of the text. Furthermore, as discussed in Section V-B3 of the manuscript, adjusting the parameter settings appropriately can further reduce the time cost. Additionally, our text protection method can be applied in offline scenarios without the need for

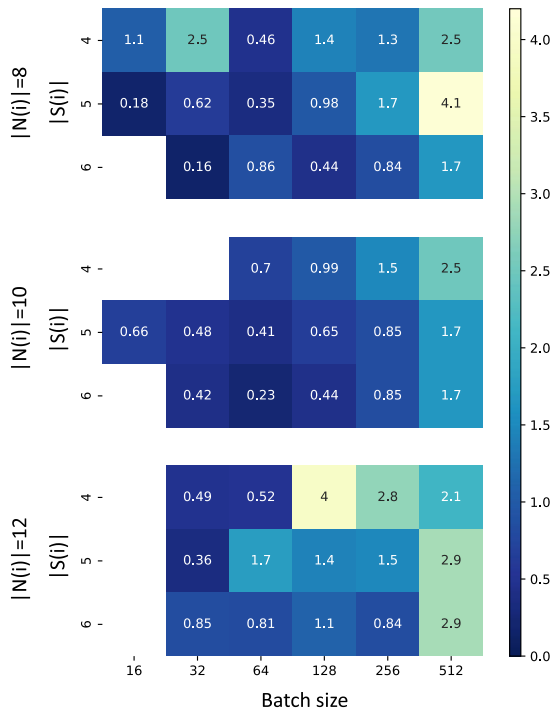


Fig. 6. The heatmap of the relationship between the replacement set size, batch size, and construction time. The masked areas indicate scenarios where, even after 100 epochs, the PSR did not reach 0.9, and the unit of the numbers in the figure in minutes.

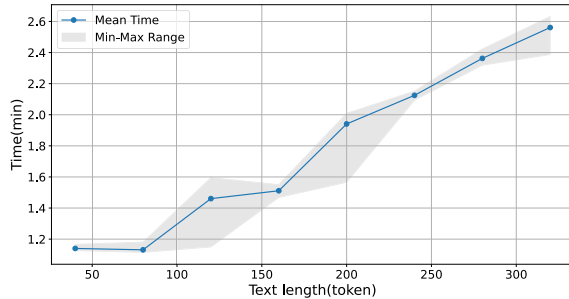


Fig. 7. The relationship between TPE construction time and text length.

real-time computation, making its application in real-world scenarios feasible.

5) *Why Truncation Works*: In Section IV-B, we mentioned that the effectiveness of constructing TPE is attributed to the tendency of dialogues to naturally end. In this section, we emphasize this point and delve deeper into the intrinsic nature of the model.

To ensure clearer contrasts in the model experimental outcomes, we opted for six prompts from the Vicuna dataset as the text examples to be protected, leveraging Vicuna as the target model. Upon inputting these prompts into the model, predictions for the next token were obtained. We selected approximately four to five tokens with probabilities close to the end token as our optimization targets to control variables. Specifically, we focused on the last dimension of the logits layer output, denoted as $output.logits[0, -1]$, and sorted these tokens in descending order. We then chose the three tokens preceding the end token and the three tokens succeeding it as our new optimization targets.

It is noteworthy that in Vicuna, we observed that the end token usually ranks second or third among all tokens. While this high ranking was somewhat unexpected, it did explain the favorable performance of the Vicuna model in our previous experiments. Additionally, in our experiments, the token ranked first consistently corresponded to token “(0x0A)”, indexed as token 13 in the dictionary, which is one of the foundational subwords used in the initialization of the vocabulary for the BBPE [44] algorithm.

Subsequently, we optimized the text to increase the probability of the model outputting the first token as the target token, iterating this process for ten rounds while maintaining other experimental settings as outlined in Section V-A. The experimental results are depicted in Figure 8.

The results indicate that optimizing for the end token and “(0x0A)” as optimization targets were easily achieved, while other tokens were less sensitive to STP’s adjustments. Notably, some optimization targets initially exhibited higher values than the end token, gradually being overtaken by the end token across the epochs, highlighting the particularity of the end token. We refer to tokens sensitive to the optimization algorithm as “sensitive tokens,” each corresponding to inherent properties of the model. For instance, the end token indicates a propensity to end dialogues, tokens ranking higher represent the model’s compliance with instructions, while tokens like “(0x0A)” correspond to certain biases generated during the model’s training. Exploring these biases may unveil deeper security implications.

VI. DISCUSSION

A. Significance of This Work and Future Directions

The proposed Silent Guardian in this paper represents the first text protection mechanism for LLMs, addressing the security gaps associated with malicious exploitation. As multimodal models and multidimensional models continue to emerge, the protective measures outlined in this paper can be extended to encompass a broader range of generative domains, including audio, images, videos, and beyond. It is anticipated that this extension will have profound implications for the field of generative large models.

B. STP Method

As shown in Figure 2, the STP method ensures good concealment while allowing for rapid convergence. Importantly, unlike the GBDA method, STP does not require introducing a reference model to guarantee concealment, ensuring its applicability across various scenarios.

On the other hand, the STP method is an optimization of the GCG method, with a notable distinction. Unlike the GCG method, STP considers the concealment of adversarial text. Given that certain online models employ input detection mechanisms to prevent malicious usage [45], traditional methods such as adding prefixes or suffixes in prompts are susceptible to perplexity detection. STP, in contrast, provides a less detectable avenue for jailbreak attacks, making it challenging to be identified.

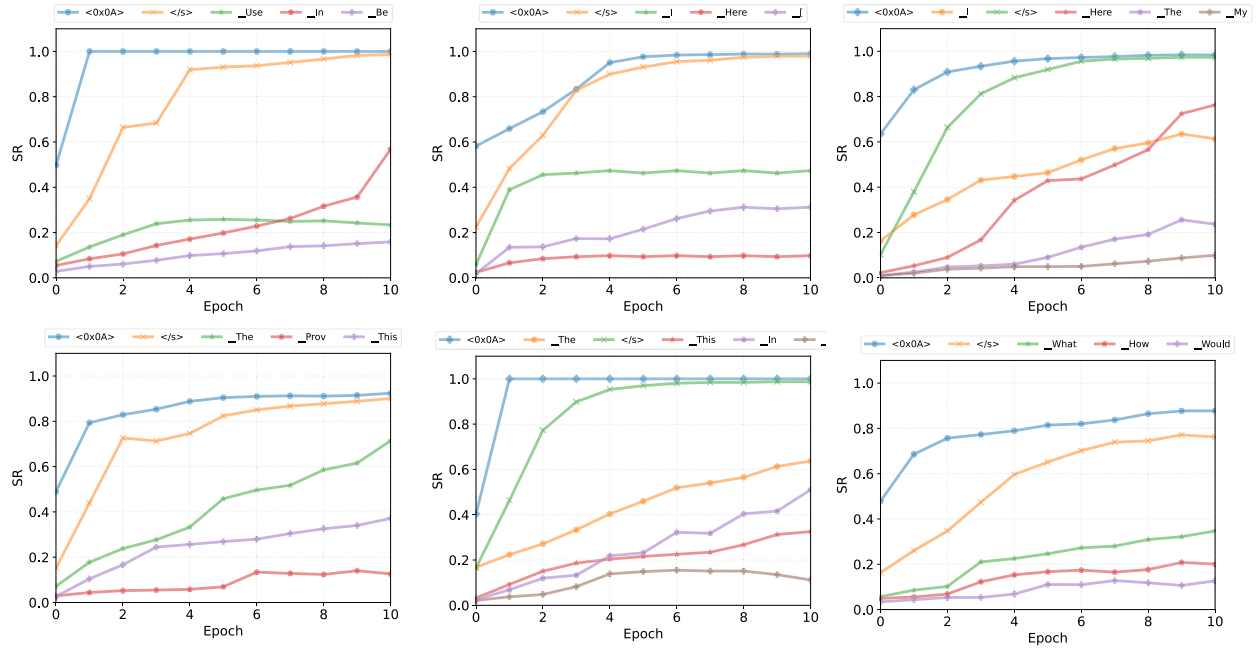


Fig. 8. The optimization result of six text examples targeting different tokens. Each line plot corresponds to the optimization results for a specific set of tokens targeted in the text. The horizontal axis Epoch represents the number of optimization rounds, and the vertical axis SR (success rate) represents the probability of the model outputting the token, calculated similarly to PSR. The legend above each plot shows, from left to right, the top five or six tokens with the highest SR rankings before optimization.

C. Truncation Protection Example

1) *Different Models*: The specified end token is not the same for different models due to tokenizer differences. For example, the LLaMA model developed by Meta uses `</s>` as the end token, which corresponds to a dictionary index of 2, while the end token in the `cl100k-base` [46] tokenizer of the GPT-4 model corresponds to a dictionary index of 100257. Therefore, different indexes should be used as the optimization target when constructing TPEs for different models.

For the work presented in this paper, achieving universality in truncation protection examples across models with different tokenizers proves challenging. Addressing this issue will be a key focus for future research efforts.

2) *Discussion on the Robustness of TPE*: In this section, we will discuss the robustness of TPE. Specifically, we selected the experimental results from Section V-B1, concerning 120-token length texts from the Novel dataset optimized on the Vicuna model and subjected them to random word deletion, random word addition, and random synonym replacement operations [47]. The randomization ratio ranged from 5% to 20%. The experimental results are presented in Table IX, where RI denotes random word insertion, RD denotes random word deletion, and SR denotes random synonym replacement, with subscripts indicating the random operation ratio.

The results indicate that the effectiveness of TPE protection deteriorates as the perturbation ratio increases. However, excessively high ratios of perturbation can cause significant information loss to the original text.

3) *Heightened Security Requirements*: For text with clearly defined security requirements, the TPE construction can be optimized by focusing on the prefixes and suffixes related to the specific topic. A new loss function can be constructed by

TABLE IX

ROBUSTNESS PERFORMANCE OF TPE

Operations	None	RI _{5%}	RI _{10%}	RI _{15%}	RI _{20%}
Metric					
PSR	0.80	0.51	0.39	0.33	0.27
Operations	None	RD _{5%}	RD _{10%}	RD _{15%}	RD _{20%}
Metric					
PSR	0.80	0.48	0.33	0.23	0.15
Operations	None	SR _{5%}	SR _{10%}	SR _{15%}	SR _{20%}
Metric					
PSR	0.80	0.51	0.37	0.29	0.25

TABLE X

TRANSFERABILITY OF PREFIX OPTIMIZATION

Prefix Metrics	Prefix ₁	Prefix ₂	Prefix ₃	Prefix ₄	Prefix ₅
PSR	0.88	0.91	0.89	0.90	0.91
PSR [†]	0.81				

introducing coefficients, and optimization should be applied only to the original text portion. For instance, when dealing with personal social media content, a defender may wish to conceal specific aspects of their lifestyle, and he can design corresponding prefixes and suffixes to construct TPE. The specific details are outlined in Algorithm 2.

In this section, we prepared five rewrite-related prefixes, denoted as Prefix₁, Prefix₂, Prefix₃, Prefix₄, and Prefix₅, along with 100 rewrite-related prefixes generated by ChatGPT-3.5. We defined the loss function as $(\sum_{i=1}^5 \text{loss}_{\text{Prefix}_i})/5$. The experimental parameters were set as $|S_i| = 5$, $|N_i| = 10$, $\text{batch size} = 128$, and $T = 15$. The experiments were conducted on the Vicuna model using texts with a length of 40 tokens from the Novel dataset. The experimental results are shown in Table X, where PSR[†] represents the average results after transferring to the 100 rewrite-related prefixes.

Algorithm 2 Truncation Protection Example for Heightened Security Requirements

Input: Original Prompt $\mathcal{P}$, Iterations T , Target Model M , Batch Size B , Loss Function $\mathcal{L}$, Prefixes $pre_1, \dots, pre_m$, Suffixes $suf_1, \dots, suf_n$

Output: Optimized prompt $\mathcal{P}$

repeat T times

for $i = 1, \dots, m$ **do**

$\mathcal{P}^i = pre_i + \mathcal{P}$

for $i = 1, \dots, n$ **do**

$\mathcal{P}^{i+m} = \mathcal{P} + suf_i$

 loss = 0

for $i = 1, \dots, m + n$ **do**

 loss += $\eta_i \cdot \mathcal{L}(\mathcal{P}^i)$

for $i = 1, \dots, len(\mathcal{P})$ **do**

$h_i = -\nabla_{v_i} \text{loss}$

$N_i = N(\mathcal{P}_i)$

$S_i = Top - k(N_i)$

$len_{part} = \frac{B}{len(\mathcal{P})}$

for $b = 1, \dots, B$ **do**

if $B > len(\mathcal{P})$ **then**

$i = \lceil \frac{b}{len_{part}} \rceil$

else

repeat

$i = \text{random}(1, len(\mathcal{P}))$

until $i \neq \text{previous } i$

$\tilde{\mathcal{P}}^{(b)} = \mathcal{P}$

$\tilde{\mathcal{P}}_i^{(b)} = \text{Uniform}(S_i)$

$\mathcal{P} = \tilde{\mathcal{P}}^{(b^*)}$, where $b^* = \arg \min_b L(\tilde{\mathcal{P}}^{(b)})$

return $\mathcal{P}$

The experimental results show that under the theme of rewriting, TPE maintains a high PSR value even after transferring to unoptimized prefixes, with an average PSR value of 0.81 after transfer.

The specific prefixes are provided in the Appendix.

4) *Discussion on TPE Text Quality*: In the aforementioned STP method, we used n-nearest neighbors of tokens in the embedding layer to construct semantically similar candidate sets. In fact, the construction of semantically similar candidate sets in STP is a replaceable module, and different methods can be used to construct semantically similar candidate sets to improve the quality of TPE texts. In this section, we mainly discuss the impact of using BERT to construct [28] these candidate sets on the quality of TPE texts.

Specifically, we used the BERT-base-uncased model to construct semantically similar sets. Compared to the original method, this method improved the semantic fluency of the text without compromising optimization efficiency, although there remains a large semantic gap compared to the original text. Some experimental results on the Vicuna model are presented in Table XI.

5) *Universal Model*: For models that share the same tokenizer but have different parameters, the approach aligns with the universal method outlined in the GCG method [20]. This

TABLE XI
EXAMPLES OF TPE ON VICUNA WITH BERT

Original Text	Protected Text
Can you help me write a resignation letter to my current employer, while leaving on good terms and expressing gratitude for the opportunities provided?	Can you exc me , a resignation letter to my previous employ es, for leaving on our terms and expressing gratitude to the opportunities provided?
Draft an apology email to a customer who experienced a delay in their order, and provide reassurance that the issue has been resolved.	Draft an apology email by a student which experienced a change in their order, and seek an assurance that his issue previously remained cleared !
Write a compelling product launch announcement email to inform our customers of our new software solution .	Write a jelling product launch announcement email to inform my staff of our current hardware deployment ?
Write a captivating movie review for a recently released science fiction film, discussing its plot, characters, and special effects.	Write a captivating feature review for a newly released action comedy film, learning about unusual situations, characters background & software engineering.

method enables simultaneous optimization for multiple models, thereby achieving enhanced TPE generality. The specific details are similar to Algorithm 2, except that the loss function calculation involves the sum across different models.

6) *Other Truncation Method*: In addition to directly generating an end token as the first token, another strategy involves guiding the model to produce refusal responses. In previous work [20], “Sure, here is” was used as an optimization prefix to guide the model during evasion attacks. Similarly, in this work, we can use refusal expressions like “I am sorry, but” as optimization targets.

D. Evasion Attack

In this section, we will discuss the impact of three potential evasion attacks on TPE.

1) *Adversarial Prompt Engineering*: Against the TPE, an attacker can construct adversarial prompts to make the LLM regain the ability to output text. For example, an attacker can add text before and after the TPE to disrupt the TPE performance.

Effective adversarial prompts can be divided into two categories, one usually requiring more complex manual construction [48], while the other uses information such as gradients to optimize the construction process [19], [20], [21], [22], [23].

For the first type of attack, the involvement of human labor makes the cost of the attack rise. If only pre-designed text is used to reach the automation goal, the attack will be less effective. The experiment in Section V-B1 is similar to this scenario, which shows that the effect of adding prompts before and after the text on the performance of TPE decreases as the length of the TPE increases, and in addition, the addition of extraneous prompts may affect the model performance.

For the second type of attack, the attacker is able to construct adversarial prefixes and suffixes through gradient

TABLE XII
REWRITE RELEVANT PREFIXES

Collected Prefixes
Prefix ₁ : Please rewrite the following text.
Prefix ₂ : Could you rephrase the following text?
Prefix ₃ : I would appreciate it if you could restate the following text.
Prefix ₄ : Kindly reformulate the following text in a different way.
Prefix ₅ : Would you mind rewording the following text?
Other Rewrite-Related Prefixes
1. Could you please express the following text in a different way?
2. I need you to paraphrase the following text.
3. Please provide a different wording for the following text.
4. Can you articulate the following text in another manner?
5. Would you mind recasting the following text?
6. Please convey the meaning of the following text in a different manner.
7. Could you put the following text into other words?
8. I require a rewording of the following text.
9. Please restate the following text using different phrasing.
10. Can you rephrase the following text in an alternative way?
11. Kindly reformulate the following text in a different manner.
12. Would you be able to reword the following text?
13. Please provide an alternative expression for the following text.
14. Could you articulate the following text in a different way?
15. I need you to restate the following text.
16. Please convey the essence of the following text in a different way.
17. Can you express the following text in another manner?
18. Would you mind rephrasing the following text?
19. Please communicate the meaning of the following text in a different manner.
20. Could you put the following text into different words?
21. I require a rephrasing of the following text.
22. Please restate the following text using alternative phrasing.
23. Can you reword the following text in an alternate way?
24. Kindly reformulate the following text in a different style.
25. Would you be able to recast the following text?
26. Please provide an alternate expression for the following text.
27. Could you articulate the following text in a different manner?
28. I need you to rewrite the following text.
29. Please convey the substance of the following text in a different way.
30. Can you express the following text in another form?
31. Would you mind rephrasing the following text?
32. Please communicate the essence of the following text in a different manner.
33. Could you put the following text into various words?
34. I require a rewording of the following text.
35. Please restate the following text using diverse phrasing.
36. Can you reword the following text in an alternative manner?
37. Kindly reformulate the following text in a different fashion.
38. Would you be able to recast the following text?
39. Please provide an alternative articulation for the following text.
40. Could you articulate the following text in a different way?
41. I need you to rephrase the following text.
42. Please convey the content of the following text in a different manner.
43. Can you express the following text in another style?
44. Would you mind rewording the following text?
45. Please communicate the substance of the following text in a different way.
46. Could you put the following text into distinct words?
47. I require a rephrasing of the following text.
48. Please restate the following text using varied phrasing.

TABLE XII
(Continued.) REWRITE RELEVANT PREFIXES

49. Can you reword the following text in an alternate manner?
50. Kindly reformulate the following text in a different manner.
51. Would you be able to recast the following text?
52. Please provide an alternate expression for the following text.
53. Could you articulate the following text in a different manner?
54. I need you to rewrite the following text.
55. Please convey the essence of the following text in a different manner.
56. Can you express the following text in another way?
57. Would you mind rephrasing the following text?
58. Please communicate the meaning of the following text in a different way.
59. Could you put the following text into different words?
60. I require a rewording of the following text.
61. Please restate the following text using alternative phrasing.
62. Can you reword the following text in an alternate way?
63. Kindly reformulate the following text in a different style.
64. Would you be able to recast the following text?
65. Please provide an alternate articulation for the following text.
66. Could you articulate the following text in a different fashion?
67. I need you to rephrase the following text.
68. Please convey the substance of the following text in a different manner.
69. Can you express the following text in another form?
70. Would you mind rewording the following text?
71. Please communicate the essence of the following text in a different way.
72. Could you put the following text into various words?
73. I require a rephrasing of the following text.
74. Please restate the following text using diverse phrasing.
75. Can you reword the following text in an alternative manner?
76. Kindly reformulate the following text in a different fashion.
77. Would you be able to recast the following text?
78. Please provide an alternative expression for the following text.
79. Could you articulate the following text in a different way?
80. I need you to rewrite the following text.
81. Please convey the content of the following text in a different way.
82. Can you express the following text in another style?
83. Would you mind rephrasing the following text?
84. Please communicate the substance of the following text in a different manner.
85. Could you put the following text into distinct words?
86. I require a rewording of the following text.
87. Please restate the following text using varied phrasing.
88. Can you reword the following text in an alternate manner?
89. Kindly reformulate the following text in a different manner.
90. Would you be able to recast the following text?
91. Please provide an alternate expression for the following text.
92. Could you articulate the following text in a different manner?
93. I need you to rewrite the following text.
94. Please convey the essence of the following text in a different manner.
95. Can you express the following text in another way?
96. Would you mind rephrasing the following text?
97. Please communicate the meaning of the following text in a different way.
98. Could you put the following text into different words?
99. I require a rewording of the following text.
100. Please restate the following text using alternative phrasing.

information, which in turn reduces the probability of sampling the end token in the first round. However, since lowering the probability of the end token does not correspond to the actual

semantics, this approach may cause the LLM responses to deviate from the original question. In addition, this type of approach usually requires a large time overhead.

2) *Simple Text Perturbation*: Against the TPE, an attacker is able to degrade the protective effect of TPE through simple text perturbations. Examples include randomly adding words to the text, randomly deleting words from the text, randomly replacing words in the text, or segmenting the text. The experiment in Section VI-C2 is similar to this scenario, and the results show that increasing the text perturbation ratio can degrade the effectiveness of TPE protection, but it also causes information loss of original text, such as structural information, semantic information of the text.

3) *Retokenization*: Retokenization refers to encoding the inputs to the model in different tokenization methods [49]. In the simplest case, we break tokens apart and represent them using multiple smaller tokens. For example, the token “studying” has a broken-token representation “study” + “ing”, among other possibilities. Through the method of retokenization, an attacker is able to reduce the adversarial nature of the TPE, thus achieving the goal of reducing its performance. However, retokenization will degrade the performance of the model to some extent.

VII. CONCLUSION

In this paper, we introduce the first text protection mechanism, SG, and the concept of TPE to address the problem of malicious exploitation of texts by LLMs. Meanwhile, we introduce the first method for constructing TPE, which is called STP. Our experimental outcomes demonstrated the effectiveness and concealment of the STP method across varied text lengths, types, and diverse models. Furthermore, TPE constructed using STP showed some level of transferability and robustness. Additionally, we explore the time overhead of constructing TPE using the STP method, which, under appropriately chosen parameters, incurs very short construction times for individual texts and exhibits a nearly linear relationship with text length. Additionally, we delved into optimizing specific token-associated model properties, which we believe can inspire future investigations into LLM characteristics. We aim to deploy SG in real-world scenarios to address the escalating concerns regarding LLM security and believe that it will find broader applications in the future.

APPENDIX

See Table XII.

REFERENCES

- [1] Y. Chang et al., “A survey on evaluation of large language models,” *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 3, pp. 1–45, 2024.
- [2] P. Liu, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, 2023.
- [3] S. I. Ross, F. Martinez, S. Houde, M. Müller, and J. D. Weisz, “The programmer’s assistant: Conversational interaction with a large language model for software development,” in *Proc. 28th Int. Conf. Intell. User Interfaces*, 2023, pp. 491–514.
- [4] Z. Xiao, X. Yuan, Q. V. Liao, R. Abdelghani, and P.-Y. Oudeyer, “Supporting qualitative analysis with large language models: Combining codebook with GPT-3 for deductive coding,” in *Proc. 28th Int. Conf. Intell. User Interfaces*, Mar. 2023, pp. 75–78.
- [5] Y. Ji, J. Han, Y. Zhao, S. Zhang, and Z. Gong, “Log anomaly detection through GPT-2 for large scale systems,” *ZTE Commun.*, vol. 21, no. 3, p. 70, 2023.
- [6] Y. Feng, J. Qiang, Y. Li, Y. Yuan, and Y. Zhu, “Sentence simplification via large language models,” 2023, *arXiv:2302.11957*.
- [7] R. Staab, M. Vero, M. Balunović, and M. Vechev, “Beyond memorization: Violating privacy via inference with large language models,” 2023, *arXiv:2310.07298*.
- [8] Lcamtuf, *Large Language Models and Plagiarism*. Accessed: Nov. 22, 2023. [Online]. Available: <https://lcamtuf.substack.com/p/large-language-models-and-plagiarism>
- [9] C. Chen and K. Shu, “Combating misinformation in the age of LLMs: Opportunities and challenges,” 2023, *arXiv:2311.05656*.
- [10] U. Khadam, M. M. Iqbal, M. A. Azam, S. Khalid, S. Rho, and N. Chilamkurti, “Digital watermarking technique for text document protection using data mining analysis,” *IEEE Access*, vol. 7, pp. 64955–64965, 2019.
- [11] X. Yang et al., “Tracing text provenance via context-aware lexical substitution,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 10, pp. 11613–11621, Jun. 2022.
- [12] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A watermark for large language models,” in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 17061–17084.
- [13] W. Qu et al., “Provably robust multi-bit watermarking for AI-generated text via error correction code,” 2024, *arXiv:2401.16820*.
- [14] I. Markwood, D. Shen, Y. Liu, and Z. Lu, “Mirage: Content masking attack against \$information - based\$ online services,” in *Proc. 26th USENIX Secur. Symp. (USENIX Security)*, 2017, pp. 833–847.
- [15] L. Zhang et al., “Enhancing privacy in large language model with homomorphic encryption and sparse attention,” *Appl. Sci.*, vol. 13, no. 24, p. 13146, Dec. 2023.
- [16] Y. Wang, R. Rajat, and M. Annavaram, “MPC-pipe: An efficient pipeline scheme for secure multi-party machine learning inference,” 2022, *arXiv:2209.13643*.
- [17] T. Shen et al., “\$SOTER\$: Guarding black-box inference for general neural networks at the edge,” in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2022, pp. 723–738.
- [18] Y. Zhang, J. Li, D. Liu, G. Chen, and J. Dou, “D-P-transformer: A distilling and probsparse self-attention rockburst prediction method,” *Energies*, vol. 15, no. 11, p. 3959, May 2022.
- [19] T. Shin, Y. Razeghi, E. Wallace, and S. Singh, “AutoPrompt: Eliciting knowledge from language models with automatically generated prompts,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Stroudsburg, PA, USA: Association for Computational Linguistics, Nov. 2020, pp. 4222–4235.
- [20] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” 2023, *arXiv:2307.15043*.
- [21] E. Wallace, S. Feng, N. Kandpal, M. Gardner, and S. Singh, “Universal adversarial triggers for attacking and analyzing NLP,” in *Proc. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, China, Stroudsburg, PA, USA: Association for Computational Linguistics, Nov. 2019, pp. 2153–2162.
- [22] C. Guo, A. Sablayrolles, H. Jégou, and D. Kiela, “Gradient-based adversarial attacks against text transformers,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Punta Cana, Stroudsburg, PA, USA: Association for Computational Linguistics, Nov. 2021, pp. 5747–5757.
- [23] Y. Wen et al., “Hard prompts made easy: Gradient-based discrete optimization for prompt tuning and discovery,” in *Proc. Adv. Neural Inf. Process. Syst.*, New Orleans, LA, USA, Dec. 2023, pp. 1–18.
- [24] J. Li, S. Ji, T. Du, B. Li, and T. Wang, “TextBugger: Generating adversarial text against real-world applications,” in *Proc. 26th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, Feb. 2019, pp. 1–15.
- [25] J. Ebrahimi, D. Lowd, and D. Dou, “On adversarial examples for character-level neural machine translation,” in *Proc. 27th Int. Conf. Comput. Linguistics (COLING)*, Santa Fe, NM, USA, Stroudsburg, PA, USA: Association for Computational Linguistics, Aug. 2018, pp. 653–663.
- [26] Y. Zang et al., “Word-level textual adversarial attacking as combinatorial optimization,” in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 6066–6080.
- [27] M. Alzantot, Y. S. Sharma, A. Elgohary, B.-J. Ho, M. Srivastava, and K.-W. Chang, “Generating natural language adversarial examples,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2018, pp. 2890–2896.

- [28] L. Li, R. Ma, Q. Guo, X. Xue, and X. Qiu, "BERT-ATTACK: Adversarial attack against BERT using BERT," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6193–6202.
- [29] S. Samanta and S. Mehta, "Towards crafting text adversarial samples," 2017, *arXiv:1707.02812*.
- [30] M. Iyyer, J. Wieting, K. Gimpel, and L. Zettlemoyer, "Adversarial example generation with syntactically controlled paraphrase networks," in *Proc. NAACL-HLT*, 2018, pp. 1875–1885.
- [31] C. Szegedy et al., "Intriguing properties of neural networks," in *Proc. 2nd Int. Conf. Learn. Represent. (ICLR)*, 2014, pp. 1–10.
- [32] X. Yang, Y. Gong, W. Liu, J. Bailey, D. Tao, and W. Liu, "Semantic-preserving adversarial text attacks," *IEEE Trans. Sustain. Comput.*, vol. 8, no. 4, pp. 583–595, Jul. 2023.
- [33] J. Jia, A. Salem, M. Backes, Y. Zhang, and N. Z. Gong, "MemGuard: Defending against black-box membership inference attacks via adversarial examples," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 259–274.
- [34] R. Shetty, B. Schiele, and M. Fritz, "\$A4NT\$: Author attribute anonymity by adversarial training of neural machine translation," in *Proc. 27th USENIX Secur. Symp. (USENIX Security)*, 2018, pp. 1633–1650.
- [35] X. Li, L. Chen, and D. Wu, "Turning attacks into protection: Social media privacy protection using adversarial attacks," in *Proc. SIAM Int. Conf. Data Min.*, 2021, pp. 208–216.
- [36] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. ICML*, 2021, pp. 8748–8763.
- [37] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10684–10695.
- [38] T. B. Brown et al., "Language models are few-shot learners," in *Proc. NeurIPS*, vol. 33, 2020, pp. 1877–1901.
- [39] W.-L. Chiang et al. *Vicuna: An Open-Source Chatbot Impressing GPT-4 With 90%* ChatGPT Quality*. Accessed: Apr. 14, 2023. [Online]. Available: <https://vicuna.lmsys.org>
- [40] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [41] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–28.
- [42] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Proc. Sov. Phys. Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [43] D. Cer et al., "Universal sentence encoder," 2018, *arXiv:1803.11175*.
- [44] C. Wang, K. Cho, and J. Gu, "Neural machine translation with byte-level subwords," *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 5, pp. 9154–9160, Apr. 2020.
- [45] G. Deng et al., "Jailbreaker: Automated jailbreak across multiple large language model chatbots," 2023, *arXiv:2307.08715*.
- [46] *Tiktoken*. Accessed: Nov. 9, 2023. [Online]. Available: <https://github.com/openai/tiktoken>
- [47] J. Wei and K. Zou, "EDA: Easy data augmentation techniques for boosting performance on text classification tasks," in *Proc. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, China, Stroudsburg, PA, USA: Association for Computational Linguistics, Nov. 2019, pp. 6383–6389.
- [48] Y. Liu et al., "Jailbreaking ChatGPT via prompt engineering: An empirical study," 2023, *arXiv:2305.13860*.
- [49] N. Jain et al., "Baseline defenses for adversarial attacks against aligned language models," 2023, *arXiv:2309.00614*.



Kejiang Chen (Member, IEEE) received the B.Eng. degree from Shanghai University (SHU) in 2015 and the Ph.D. degree from the University of Science and Technology of China (USTC) in 2020. He is currently working as an Associate Research Fellow with USTC. His research interests include information hiding and AI security.



Xiaojian Yuan received the B.Eng. degree from China University of Mining and Technology (CUMT) in 2021. He is currently pursuing the Ph.D. degree with the University of Science and Technology of China. His research interests include trustworthy AI.



Yuang Qi received the B.Eng. degree from the University of Science and Technology of China (USTC) in 2021, where he is currently pursuing the Ph.D. degree in cyberspace security. His research interests include steganography, watermarking, and AI security.



Weiming Zhang received the M.S. and Ph.D. degrees from Zhengzhou Information Science and Technology Institute, China, in 2002 and 2005, respectively. He is currently working as a Professor with the School of Information Science and Technology, University of Science and Technology of China. His research interests include information hiding and multimedia security.



Jiawei Zhao received the B.Eng. degree from the University of Science and Technology of China (USTC) in 2024, where he is currently pursuing the degree. His research interests include large language model security and deep learning.



Nenghai Yu received the B.Eng. degree from Nanjing University of Posts and Telecommunications in 1987, the M.E. degree from Tsinghua University in 1992, and the Ph.D. degree from the University of Science and Technology of China in 2004. He is currently working as a Professor with the University of Science and Technology of China. His research interests include multimedia security, multimedia information retrieval, video processing, and information hiding.